

Llenguatges de Programació

6 de juny de 2025

Grau en Enginyeria Informàtica

Gerard Escudero, Jordi Petit
Facultat d'Informàtica de Barcelona
Universitat Politècnica de Catalunya

© 2025 Universitat Politècnica de Catalunya, 2025 (UPC). Tots els drets reservats. Aquest examen no pot ser publicat, compartit, reproduït o distribuït sense el consentiment exprés per escrit de la UPC.

Instruccions

L'examen dura dues hores i mitja. Poseu el vostre nom a cada full. Contesteu cada problema en fulls diferents. Es valorarà la concisió, claredat i brevetat a més de la completesa i l'exactitud de les respostes. Feu bona lletra.

1 λ -càlcul

(2 punts)

Recordeu que en λ -càlcul:

$$\begin{aligned} \mathbf{2} &\equiv \lambda s.\lambda z.s(sz) \\ \mathbf{4} &\equiv \lambda s.\lambda z.s(s(s(sz))) \\ \mathbf{mul} &\equiv \lambda m.\lambda n.\lambda f.n(mf) \end{aligned}$$

1. Doneu una definició per la funció **doble** en λ -càlcul.
2. Comproveu que el resultat d'avaluar **doble 2** en ordre normal és **4**.

2 Inferència de tipus i raonament equacional

(3 punts)

Recordeu els tipus d'aquests operadors i funcions habituals en Haskell:

$(\gg=)$:: **Monad** $m \Rightarrow m\ a \rightarrow (a \rightarrow m\ b) \rightarrow m\ b$
return :: **Monad** $m \Rightarrow a \rightarrow m\ a$
 $(-)$:: **Num** $a \Rightarrow a \rightarrow a \rightarrow a$
div :: **Integral** $a \Rightarrow a \rightarrow a \rightarrow a$

1. Inferiu el tipus complet de l'expressió:

$(\gg= mg)$

amb

$m = (\text{div } 2)$ -- Per exemple: $m\ 7$ val 3
 $mg\ x$
| **even** x = **return** $(m\ (x - 1))$
| **otherwise** = **return** $(m\ x)$

2. Detalleu, pas a pas, l'avaluació d'aquestes dues expressions:

Just $4 \gg= mg$
 $[5..8] \gg= mg \gg= mg$

3 Mònada estat

(2'5 punts)

Recordeu l'examen parcial? En aquell examen calia implementar una funció per llegir un arbre binari a partir d'una llista.

A continuació es dona una possible implementació per llegir un arbre binari de naturals donat a través d'una llista que correspon al seu recorregut en preordre, amb valors negatius per indicar els arbres buits. Fixeu-vos que la funció *llegirAux* retorna una parella amb l'arbre llegit i la resta de la llista pendent de llegir.

```
data Arbin = Buit | Node Int Arbin Arbin
```

```
llegir  :: [Int] → Arbin  
llegir = fst . llegirAux
```

```
llegirAux :: [Int] → (Arbin, [Int])  
llegirAux (x:xs)  
  | x < 0 = (Buit, xs)  
  | x ≥ 0 = (Node x fe fd, rest)  
    where  
      (fe, tmp) = llegirAux xs  
      (fd, rest) = llegirAux tmp
```

Per altra banda, recordeu que la mònada estat ofereix:

- el tipus

```
data State s a = s → (a, s)
```

que permet definir la monàda estat (*State TipusEstat TipusResultat*),

- les funcions de treball

```
get      :: MonadState s m ⇒ m s      -- obtenir l'estat  
put      :: MonadState s m ⇒ s → m ()  -- actualitzar l'estat  
return   :: MonadState s m ⇒ a → m a   -- retornar el valor
```

que permeten escriure funcions amb manipulació d'estats (usualment amb notació **do**),

- la funció de crida

```
runState  :: State s a → s → (a, s)
```

que, donada una funció amb manipulació d'estat i un estat inicial, torna una tupla amb el valor de retorn i el nou estat.

Reimplementeu *llegir* i *llegirAux* usant la mònada estat i notació **do** per obtenir un codi clar i net.

4 Python

(2'5 punts)

Responeu breument aquestes preguntes sobre Python.

1. Digueu què escriu aquest programa:

```
def create_multipliers ():  
    return [lambda x: i * x for i in range(4)]  
  
multipliers = create_multipliers ()  
print([m(2) for m in multipliers ])
```

2. Digueu què fa aquest programa:

```
def f ():  
    x = 42  
    def g ():  
        x = x + 1  
        return x  
    return g()  
  
print(f())
```

3. Expliqueu com es resolen els conflictes al cridar mètodes deguts a l'herència múltiple en Python i digueu què escriu aquest programa:

```
class A:  
    def m(self): return 'A'  
    def f(self): return 'A: ' + self.m()  
  
class B:  
    def m(self): return 'B'  
    def f(self): return 'B: ' + self.m()  
  
class C(A, B):  
    def m(self): return 'C'  
  
c = C()  
print(c.f())
```

4. Doneu una implementació amb tècnica de *continuation passing style* d'una funció per calcular el perímetre d'un cercle donat el seu radi i doneu-ne un exemple d'ús per escriure el perímetre d'un cercle de radi 3.
5. Dissenyeu un decorador que validi que tots els paràmetres enters d'una funció siguin estrictament positius. Si algun paràmetre no compleix les restriccions, ha de llançar una excepció *ValueError*. Doneu-ne un exemple d'ús.